

RAPTOR-HUNTY

Stealth Recon & Vulnerability Assessment Framework

Complete Code Documentation & Technical Reference

Metasploit Framework · Nmap · Bash · Ruby

Author	Andrew Alba
Project	CyberSecurity Lab
Version	v1.0 — May 2026
Platform	Kali Linux / Metasploit Framework
License	MIT — Ethical Hacking
GitHub	github.com/andrewAlba/Pentest-Automation-Toolkit

1. Project Overview

Raptor-Hunty is an automated penetration testing assistant built for controlled lab environments. It wraps Metasploit's msfconsole and Nmap into a guided interactive shell, walking the analyst through the complete recon-to-report pipeline without manual RC file authoring. Developed as part of the MetalBin S.A. Industrial Security Lab ethical hacking project.

What it does — step by step

1	Displays a centered ASCII art banner with the tool name and author.
2	Asks the user to choose language: Spanish or English.
3	Collects LHOST (attacker IP) and RHOSTS (target network) with input validation.
4	Asks for target OS type: Windows, Linux, macOS, Android, or Auto.
5	Asks for noise level: Noisy (-T4), Semi-noisy (-T3), or Quiet (-T2 stealth).
6	Launches msfconsole with the RC file and runs db_nmap with the chosen flags.
7	Displays discovered hosts, services, and vulnerabilities from the MSF database.
8	Suggests Metasploit auxiliary modules and CVE references per detected service.
9	Writes a structured report file to the Desktop automatically.
10	Sends a desktop notification on completion if notify-send is available.

2. Installation & Setup

Requirements

- Kali Linux (or any distro with Metasploit Framework installed)
- msfconsole, nmap, whiptail, sha512sum — all pre-installed on Kali
- PostgreSQL running and connected to Metasploit (run: msfdb init)

Step-by-step setup

Step 1 — Create project folder

```
mkdir -p ~/lab_msf
cd ~/lab_msf
```

→ Creates the working directory where all tool files will live.

Step 2 — Create authorized users list

```
nano authorized_users.txt
# Type your system username (e.g. kali) and save
```

→ Only users listed here can run the tool. Prevents accidental execution by other users.

Step 3 — Create run.sh

```
nano run.sh
# Paste the full Bash script (see Section 3)
# Save: Ctrl+O | Exit: Ctrl+X
```

→ The main entry point. Handles all user interaction before launching msfconsole.

Step 4 — Create lab_menu.rc

```
nano lab_menu.rc
# Paste the full RC/Ruby script (see Section 4)
# Save: Ctrl+O | Exit: Ctrl+X
```

→ The Metasploit resource file. Contains scan logic, module suggestions, and report writer.

Step 5 — Generate integrity hash

```
sha512sum lab_menu.rc > lab_menu.rc.sha512
```

→ Creates a SHA-512 fingerprint. run.sh verifies this hash before every execution to detect tampering.

Step 6 — Set execute permission

```
chmod +x run.sh
```

→ Makes run.sh executable. Without this, Bash will refuse to run the script.

Step 7 — Launch the tool

```
./run.sh
```

→ Starts Raptor-Hunty. Follow the interactive prompts in the terminal.

3. run.sh — Full Code with Explanation

This is the Bash entry point. It handles user interaction, security checks, and launches msfconsole. Each block shows the code followed by a plain-English explanation.

Block 1 — Shebang, strict mode, and file paths

```
#!/bin/bash
set -euo pipefail

SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
RC_FILE="$SCRIPT_DIR/lab_menu.rc"
HASH_FILE="$SCRIPT_DIR/lab_menu.rc.sha512"
AUTH_FILE="$SCRIPT_DIR/authorized_users.txt"
DESKTOP_DIR="$HOME/Desktop"
REPORT_FILE="$DESKTOP_DIR/raptor_report.txt"
LOG_FILE="$DESKTOP_DIR/raptor_log.txt"
export LC_ALL=C
```

→ `#!/bin/bash` tells the OS to use Bash. `set -euo pipefail` is a safety net: `-e` stops on any error, `-u` treats unset variables as errors, `-o pipefail` catches failures in pipes. `SCRIPT_DIR` finds the real folder of `run.sh` regardless of where you call it from. All paths are built relative to it so the tool works from any location.

Block 2 — `center_text()` and `show_banner()`

```
center_text() {
    local text="$1"
    local cols
    cols=$(tput cols 2>/dev/null || echo 80)
    local len=${#text}
    local pad=$(( (cols - len) / 2 ))
    (( pad < 0 )) && pad=0
    printf "%*s%s\n" "$pad" "" "$text"
}

show_banner() {
    clear
    echo -e "\e[31m"
    center_text " ____"
    center_text "/  )"
    center_text ".-^^-^-"
    center_text "Raptor-Hunty Modo Furtivo"
    center_text "Andrew Alba"
    echo -e "\e[0m"
}
```

→ `center_text()` reads the terminal width with `tput cols` (defaults to 80 if unavailable), calculates left padding to center the string, and prints it with `printf`. `show_banner()` calls `clear` to wipe the screen, switches the color to red (`\e[31m`), draws the ASCII raptor art line by line, then resets with `\e[0m`.

Block 3 — IP and CIDR input validation

```
valid_ip() {
    [[ $1 =~ ^([0-9]{1,3}\.){3}[0-9]{1,3}$ ]]
}

valid_cidr() {
    [[ $1 =~ ^([0-9]{1,3}\.){3}[0-9]{1,3}/[0-9]+$ ]]
}
```

→ Both functions use Bash regex matching ([[=~]]). `valid_ip()` checks for the pattern X.X.X.X. `valid_cidr()` additionally requires a /prefix at the end (e.g. 192.168.10.0/24). They return 0 (true) on match and 1 (false) on no match — used directly in while loops to keep asking until the user enters valid data.

Block 4 — Language selection and string loading

```
choose_language() {
    echo "1) Espanol"
    echo "2) English"
    read -r -p "> " opt
    case "$opt" in
        2) LANGUAGE="en" ;;
        *) LANGUAGE="es" ;;
    esac
}

load_lang() {
    case "${LANGUAGE:-es}" in
        es)
            TITLE="Laboratorio Metasploit"
            ASK_LHOST="Tu IP local (LHOST): "
            STARTING="Iniciando..."
            DONE="Proceso finalizado" ;;
        en)
            TITLE="Metasploit Lab"
            ASK_LHOST="Your local IP (LHOST): "
            STARTING="Starting..."
            DONE="Process finished" ;;
    esac
}
```

→ `choose_language()` shows the menu and sets `$LANGUAGE` to 'en' or 'es'. `load_lang()` then populates all UI string variables based on that choice. `${LANGUAGE:-es}` means "use `$LANGUAGE`, or default to es if it is not set". `load_lang()` is called twice: once at startup for the default language, and again after the user picks.

Block 5 — choose_input() — collecting scan parameters

```
choose_input() {
    # Loop until a valid attacker IP is entered
```

```

while true; do
    read -r -p "$ASK_LHOST" LHOST
    valid_ip "$LHOST" && break
    echo "[!] Invalid IP"
done

# Loop until a valid target CIDR is entered
while true; do
    read -r -p "$ASK_RHOSTS" RHOSTS
    valid_cidr "$RHOSTS" && break
    echo "[!] Invalid CIDR"
done

# OS selection (defaults to 5=auto on invalid input)
read -r -p "> " OS_MODE
case "$OS_MODE" in 1|2|3|4|5) ;; *) OS_MODE=5 ;; esac

# Noise level (defaults to 2=semi-noisy on invalid input)
read -r -p "> " NOISE_MODE
case "$NOISE_MODE" in 1|2|3) ;; *) NOISE_MODE=2 ;; esac

# Translate noise level to actual Nmap flags
case "$NOISE_MODE" in
    1) NMAP_OPTS="-sV -O -A --osscan-limit -T4 --script vuln" ;;
    2) NMAP_OPTS="-sV -O --osscan-limit -T3 --script vuln" ;;
    3) NMAP_OPTS="-sS -Pn -n -T2" ;;
esac
}

```

→ The while loops keep asking until the user enters a valid IP or CIDR — no bad data reaches Nmap. OS_MODE and NOISE_MODE use case statements with a wildcard (*) fallback to a safe default. The final case block translates the noise level (1/2/3) into actual Nmap command-line flags that will be passed to db_nmap inside Metasploit.

Block 6 — check_access() and check_files() — security gates

```

check_access() {
    user="$(whoami)"
    if [[ -f "$AUTH_FILE" ]]; then
        grep -qx "$user" "$AUTH_FILE" || {
            echo "[!] Unauthorized user: $user"
            exit 1
        }
    fi
}

check_files() {
    [[ -f "$RC_FILE" ]] || { echo "[!] Missing: $RC_FILE"; exit 1; }
    [[ -f "$HASH_FILE" ]] || { echo "[!] Missing: $HASH_FILE"; exit 1; }
    sha512sum -c "$HASH_FILE" # Fails if RC file was modified
}

```

→ check_access() gets the current username with whoami and checks it against authorized_users.txt using grep -qx (exact full-line match, quiet mode). If not found, the script exits with code 1.

check_files() first confirms both files exist, then runs sha512sum -c which re-hashes lab_menu.rc and compares it to the stored hash — any modification causes an immediate exit.

Block 7 — progress_bar() and main() — orchestration

```
progress_bar() {
    while true; do
        for p in 5 15 25 35 45 55 65 75 85 95 100; do
            echo $p
            sleep 1
        done
    done
}

main() {
    load_lang          # Load default strings
    check_access       # Verify authorized user
    check_files        # Verify RC file integrity
    show_banner
    choose_language    # User picks ES or EN
    load_lang          # Reload in chosen language
    show_banner
    choose_input       # Collect all scan parameters

    # Export all params as env vars for lab menu.rc
    export LANGUAGE LHOST RHOSTS OS_MODE NOISE_MODE NMAP_OPTS REPORT_FILE

    # Start whiptail progress gauge in background
    { progress_bar; } | whiptail --gauge "Raptor is tracking..." 8 60 0 &
    GAUGE_PID=$!

    # Run msfconsole and log all output
    msfconsole -q -r "$RC_FILE" | tee "$LOG_FILE"

    kill $GAUGE_PID 2>/dev/null || true
    echo "[+] $DONE"
    echo "[+] Report saved: $REPORT_FILE"
    command -v notify-send >/dev/null && notify-send "Raptor" "$DONE"
}

main "$@"
```

→ progress_bar() runs endlessly in a subshell, echoing numbers 5-100 every second. whiptail --gauge reads those numbers from stdin and renders a visual progress bar. The & sends it to the background and \$! captures its PID so we can kill it when the scan ends. msfconsole -q suppresses the startup banner; -r loads the RC file directly. tee writes output to both the terminal and the log file simultaneously.

4. lab_menu.rc — Full Code with Explanation

This file is loaded by msfconsole via the -r flag. It mixes native Metasploit console commands with embedded Ruby blocks (<ruby>...</ruby>) that access the MSF framework API directly.

Block 1 — Ruby header: validate environment variables

```
<ruby>
require 'ipaddr'

# Returns true if string is a valid IP or CIDR block
def valid_ip_or_cidr?(s)
  begin
    IPAddr.new(s)
    true
  rescue
    false
  end
end

# Read all parameters from environment variables
lhost      = ENV['LHOST'].to_s
rhosts     = ENV['RHOSTS'].to_s
os_mode    = ENV['OS_MODE'].to_s
noise_mode = ENV['NOISE_MODE'].to_s
report_file = ENV['REPORT_FILE'].to_s
nmap_opts  = ENV['NMAP_OPTS'].to_s

# Exit immediately if IPs are invalid
unless valid_ip_or_cidr?(lhost)
  print_error('LHOST invalid'); exit
end
unless valid_ip_or_cidr?(rhosts)
  print_error('RHOSTS invalid'); exit
end

# Map numeric selections to human-readable strings
selected_os = case os_mode
  when '1' then 'windows'
  when '2' then 'linux'
  when '3' then 'macos'
  when '4' then 'android'
  else 'auto'
end

print_status("OS: #{selected_os} | LHOST: #{lhost} | RHOSTS: #{rhosts}")
</ruby>
```

→ Ruby's IPAddr.new() raises an exception on invalid input — the rescue block catches it and returns false. ENV[] reads the shell environment variables exported by run.sh. print_status() / print_error() are Metasploit API methods that format output with [*] and [!] prefixes.

Block 2 — MSF console commands: database setup and scan

```
# Start logging the full session
spool /tmp/raptor_msf.log

# Verify PostgreSQL is connected
db_status

# Clear previous session data (fresh start)
hosts -d
services -d
vulns -d
notes -d
creds -d
loot -d

# Run Nmap AND import results into MSF database automatically
db_nmap <%= ENV['NMAP_OPTS'] %> <%= ENV['RHOSTS'] %>

# Display everything found
hosts
services
vulns

# Export full database snapshot as XML
db_export -f xml /tmp/reporte_lab.xml
```

→ spool redirects all console output to a file for review later. The -d flags on hosts/services/vulns delete old records so each run starts clean. db_nmap is Metasploit's Nmap wrapper — it runs the scan AND automatically imports results into PostgreSQL. The <%= %> ERB tags inject the environment variable values at runtime before the RC file is executed.

Block 3 — Ruby: automatic module suggestions per service

```
<ruby>
print_status('Suggested modules for review:')

# Loop over every service found in the MSF database
framework.db.services.each do |s|
  port = s.port.to_i
  name = s.name.to_s.downcase

  print_line(" - #{s.host.address}:#{port}/#{s.proto} #{name}")

  if name.include?('ssh')
    print_line('   -> auxiliary/scanner/ssh/ssh_version')
    print_line('   -> possible CVE: check OpenSSH version')
  elsif name.include?('http')
    print_line('   -> auxiliary/scanner/http/http_version')
    print_line('   -> possible CVE: check web server version')
  elsif name.include?('smb') || port == 445
    print_line('   -> auxiliary/scanner/smb/smb_version')
    print_line('   -> possible CVE: CVE-2017-0144 / MS17-010')
  elsif name.include?('ftp')
```

```

    print_line('    -> auxiliary/scanner/ftp/ftp_version')
  elsif name.include?('rdp') || port == 3389
    print_line('    -> auxiliary/scanner/rdp/rdp_scanner')
    print_line('    -> possible CVE: check NLA configuration')
  else
    print_line('    -> manual review based on version/banner')
  end
end
end
</ruby>

```

→ framework.db.services returns all services stored in the MSF database after the scan. Each service object has .port, .name, .proto, and .host attributes. The if/elsif chain matches the service name string or port number to known service types and prints the appropriate MSF auxiliary module path and CVE hint for the analyst to follow up.

Block 4 — Ruby: write the final report file

```

<ruby>
report = ENV['REPORT_FILE'].to_s

begin
  File.open(report, 'w') do |f|
    f.puts '==== RAPTOR REPORT ==== '
    f.puts "Date: #{Time.now}"
    f.puts "LHOST: #{ENV['LHOST']}"
    f.puts "RHOSTS: #{ENV['RHOSTS']}"
    f.puts
    f.puts 'Hosts:'
    framework.db.hosts.each { |h| f.puts " - #{h.address}" }
    f.puts
    f.puts 'Services:'
    framework.db.services.each { |s| f.puts " - #{s.host.address}:#{s.port}/#{s.proto}" }
  }

  f.puts
  f.puts 'Vulnerabilities:'
  framework.db.vulns.each { |v| f.puts " - #{v.host.address} #{v.name}" }
end
print_good("Report saved: #{report}")
rescue => e
  print_error("Could not write report: #{e}")
end
</ruby>

spool off
echo [*] Done.
jobs

```

→ File.open with 'w' creates or overwrites the report file. The block pattern (do |f| ... end) ensures the file handle is closed automatically even if an error occurs. framework.db.hosts / .services / .vulns query the live MSF PostgreSQL database. print_good() outputs a [+] green-prefixed line. The rescue block catches file permission or disk errors gracefully. spool off closes the session log. jobs shows any background MSF jobs still running.

5. Quick Reference Tables

Noise Level Reference

Level	Nmap Flags	Use Case
Noisy (1)	<code>-sV -O -A --osscan-limit -T4 --script vuln</code>	Full fingerprint, fast. Generates IDS alerts. Best for isolated labs.
Semi-noisy (2)	<code>-sV -O --osscan-limit -T3 --script vuln</code>	Balanced. Default mode. Slower, fewer alerts.
Quiet (3)	<code>-sS -Pn -n -T2</code>	Stealth SYN scan. No ping. No DNS. Avoids detection.

Service to Module Mapping

Service	Suggested MSF Module	CVE Reference
SSH	<code>auxiliary/scanner/ssh/ssh_version</code>	OpenSSH version-dependent
HTTP/HTTPS	<code>auxiliary/scanner/http/http_version</code>	Web server version-dependent
SMB (445)	<code>auxiliary/scanner/smb/smb_version</code>	CVE-2017-0144 / MS17-010
FTP	<code>auxiliary/scanner/ftp/ftp_version</code>	FTP daemon version-dependent
RDP (3389)	<code>auxiliary/scanner/rdp/rdp_scanner</code>	NLA configuration review
Other	(manual review)	Check banner and version string

Output Files

File	Location	Content
<code>raptor_report.txt</code>	<code>~/Desktop/</code>	Structured hosts, services, vulns summary
<code>raptor_log.txt</code>	<code>~/Desktop/</code>	Full msfconsole session output

6. Legal Disclaimer & Author

NOTE: This tool is built for authorized lab environments only. Running network scans against systems you do not own or have explicit written permission to test is illegal in most jurisdictions. The author takes no responsibility for misuse. Always operate within the scope of a signed Rules of Engagement document.

Name	Andrew Alba
Role	Cybersecurity Ethical Hacking Network Security
Project	cyberSecurity Lab — Practical Pentesting Project
Tool	Raptor-Hunty v1.0
Date	May 2026
GitHub	github.com/andrewAlba/Pentest-Automation-Toolkit